

UNIVERSITY OF TEXAS AT AUSTIN, ECE 394

Control of Power Electronics – Final Report

Instructor: Dr. BRIAN JOHNSON



Alex Jang, Tanya Nikam, Luis Roman

Spring 2026

Contents

1	Introduction	3
1.1	System Overview	3
1.2	Theory of Operation	4
2	Experiments and Implementation	6
2.1	Control Implementation and Approach	6
2.2	Experimental Procedure and Results	8
2.2.1	Boost Open Loop	8
2.2.2	Boost Closed Loop	11
2.2.3	Open Loop Motor Drive	12
2.2.4	Closed Loop Motor Control	13
3	Conclusion	14
3.1	Discussion of Results	14
3.2	Reflection	14
3.2.1	Takeaways	14
3.2.2	Future Work	14

1 Introduction

The objective of this project was to design, implement, and test the control of an e-bike motor drive built around a two-stage power electronics system. The first stage is a boost converter that steps up a battery-level DC input to a higher DC voltage. The second stage is a three-phase voltage source inverter, integrated into an Integrated Power Module (IPM), that drives an AC motor mounted in a bike trainer. Control is executed on a Texas Instruments C2000 microcontroller, which generates PWM signals, samples currents and voltages through the on-chip ADC, decodes Hall sensor and throttle signals, and runs the control loops in an interrupt service routine (ISR).

Our team's technical objective was to get as much of the overall system working as possible. First, bring the boost converter from open loop verification to closed loop current control with reference tracking. Second, bring the motor drive from open loop three-phase excitation to a closed loop i_d/i_q current controller whose reference is set by the throttle input. Both stages reuse the modular C code structure developed across the homework assignments and the simulation environment from earlier in the course.

1.1 System Overview

The system is composed of a 24V battery feeding a 24V-48V Boost converter operating at 375kHz. This boost converter fed power to the Integrated Power Module operating at 15kHz. The integrated power module fed 3-phase currents to the motor inside the wheel. The wheel was set on bike stand so it could spin freely.

Parallel to these systems is the C2000 microcontroller which is running a variety of different tasks. The C2000 was responsible for sensing the different key voltages and currents in order to control the duty ratios of all the switches in order to operate at optimum efficiency and to track throttle accurately.

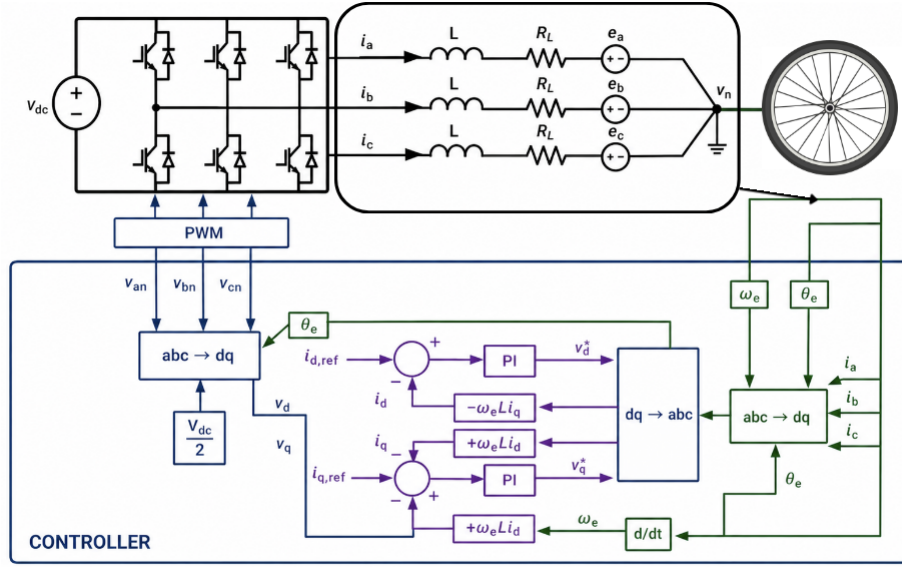


Figure 1.1: Block Diagram of System, Control -> Motor

1.2 Theory of Operation

There are certain equations that are essential to understanding the system.

For the motor, we need to relate the i_q we control to the actual torque applied to the wheel. This is shown below:

$$P_{dq} = P_{mech}$$

$$\frac{3}{2} [e_d i_d + e_q i_q] = \tau_e \omega_m$$

Since $e_d = 0$ in this context:

$$\frac{3}{2} e_q i_q = \tau_e \omega_m$$

Substituting $\omega_m = \frac{2}{p} \omega_e$ and $e_q = \lambda_m \omega_e$:

$$\frac{3}{2} \lambda_m \omega_e i_q = \tau_e \frac{2}{p} \omega_e$$

$$\tau_e = \frac{3}{2} \lambda_m \frac{p}{2} i_q$$

We also need to understand the dq transform, which is explained in the control implementation section.



Figure 1.2: Fully assembled PCB used for testing

2 Experiments and Implementation

2.1 Control Implementation and Approach

Throughout all of the different control loops PI controllers were used. Both were controlling current using Kp and Ki. However, depending on the application, they were implemented differently.

For the boost controller, we are aiming to regulate our inductor current to a certain value. For full operation of the 24V to 48V boost, we expect to see 10A of current. However, since we ran it at lower output, we aimed to keep the inductor current roughly around 1.5A. The PI controller implemented uses pole-zero cancellation to reduce noise and push a changing duty cycle to the MOSFETs. Before working on hardware, we implemented current control on a PLECS simulation as shown below in Figure 2.1. Figure 2.2 shows the ideal tracking behavior we want to get when we change the reference value.

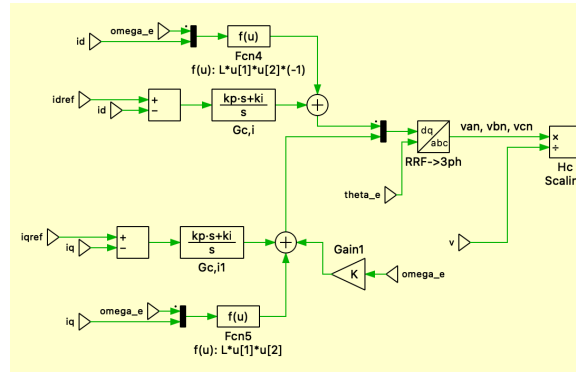


Figure 2.1: Current Control Simulation on PLECS

$$k_i = \frac{R}{\tau_i}, \quad k_p = \frac{L}{\tau_i}, \quad \tau_i = \frac{1}{2\pi \cdot \frac{f_{sw}}{10}}$$

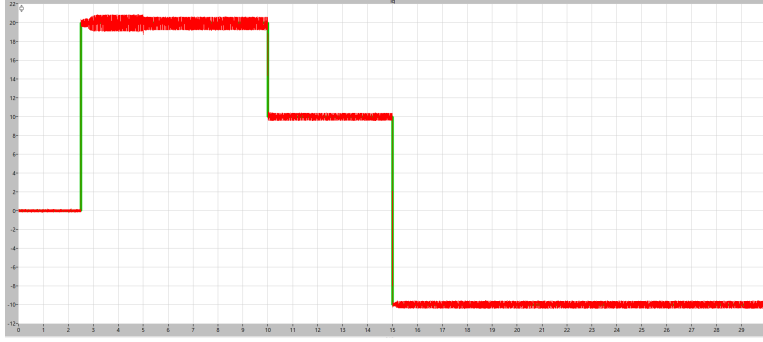


Figure 2.2: Closed Loop System Tracking Current

For the motor controller, we implemented a PI controller to regulate our i_q . In relation to the full motor system, the PI controller lives within the dq domain and acts on the equivalent 3 phase current after it has been transformed. Just as in the boost, we use pole-zero cancellation to keep our system around the reference i_q . However, rather than i_q being a physical current that can be directly measured, i_q is used to apply electrical torque in the machine model. By controlling i_q , we are able to control the rotation of the motor, either allowing for steady speed or a changing speed depending on throttle input.

Listing 2.1: Modular C Code for PI Control and DQ Transform

```
// --- abc -> dq (Park Transform) ---
id = (2.0/3.0) * (iLA * cos(theta_e) + iLB * cos(theta_e - OFFSET)
    + iLC * cos(theta_e + OFFSET));
iq = (2.0/3.0) * (-iLA * sin(theta_e) - iLB * sin(theta_e - OFFSET)
    - iLC * sin(theta_e + OFFSET));

// --- Current error ---
id_err = id_ref - id; iq_err = iq_ref - iq;

// --- PI controller (trapezoidal integration) ---
d_int = d_int_prev + Ki * (Ts/2.0) * (id_err + id_err_prev);
q_int = q_int_prev + Ki * (Ts/2.0) * (iq_err + iq_err_prev);

// --- dq voltage with decoupling ---
vd = d_int + Kp * id_err + omega_e * Ls * iq;
vq = q_int + Kp * iq_err - omega_e * Ls * id;
// --- Normalize to modulation indices ---
md = vd * (2.0 / Vdc);
mq = vq * (2.0 / Vdc);

// --- dq -> abc (Inverse Park) ---
m_a = md * cos(theta_e) - mq * sin(theta_e);
m_b = md * cos(theta_e - OFFSET) - mq * sin(theta_e - OFFSET);
m_c = md * cos(theta_e + OFFSET) - mq * sin(theta_e + OFFSET);
```

Ideally, we want our current waveforms to be balanced in our 3-Phase system, matching these results we got from our simulation.

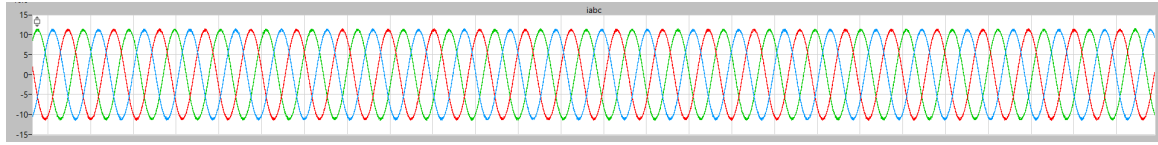


Figure 2.3: Ideal simulated 3-Phase Current Waveforms

Similarly, these are the expected Hall signals from the simulation

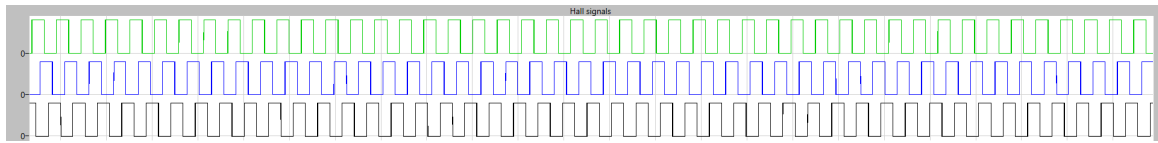


Figure 2.4: Ideal Hall Waveforms

2.2 Experimental Procedure and Results

2.2.1 Boost Open Loop

The first step was to finish the hardware assembly, so we soldered on various components onto the PCB, including the IPM, test points, current sensor, header pins, etc. Next, we verified operation of the on board power supplies (15V, 5V, and 3.3V). On our first initial power up check, we observed a short from 5V to ground. To fix this, we desoldered the power supply and adjacent coupling capacitors. Instead of replacing the module, we used the C2000 itself to supply 5V to the system.

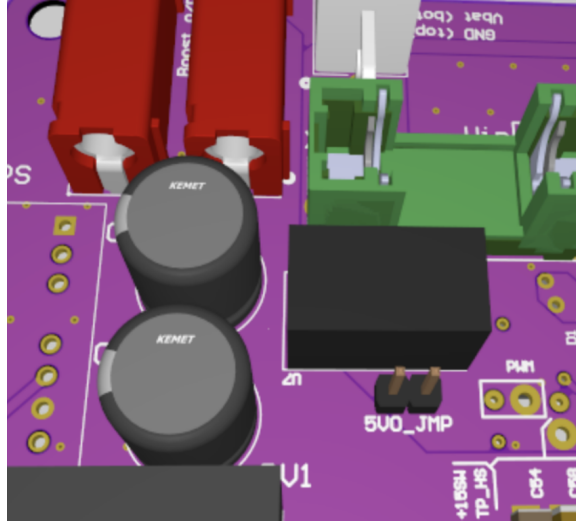


Figure 2.5: 5V Power Supply: Bypassed with C2000

The next time we applied a 24V input to the board, we noticed a rapid clicking noise. To fix this, we desoldered R10 and R11 to disable the gate driver.

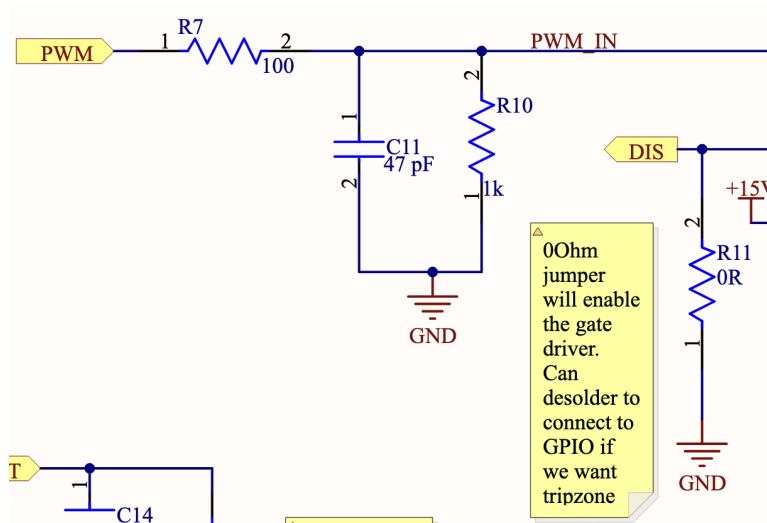


Figure 2.6: Desoldered 0 Ohm Resistors

After fixing that, we disabled the IPM stage and once again tried our boost converter bring-up and it worked. We initially applied an 85% Duty Cycle to the PWM Pin (59) that drove the boost stage. We manually sent 28V volts through the output verified the the PWM + highside and lowside switch nodes. These results, included below (figures 2.7 and 2.8) matched what we would expect to see for those waveforms.

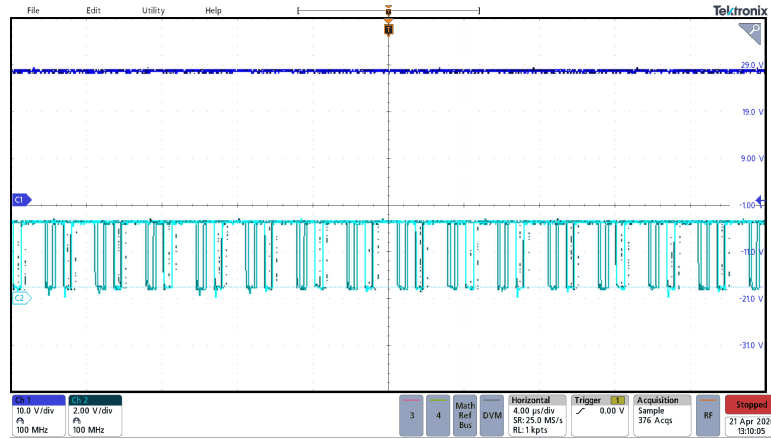


Figure 2.7: Successful High-side and Low-side Switching Operation

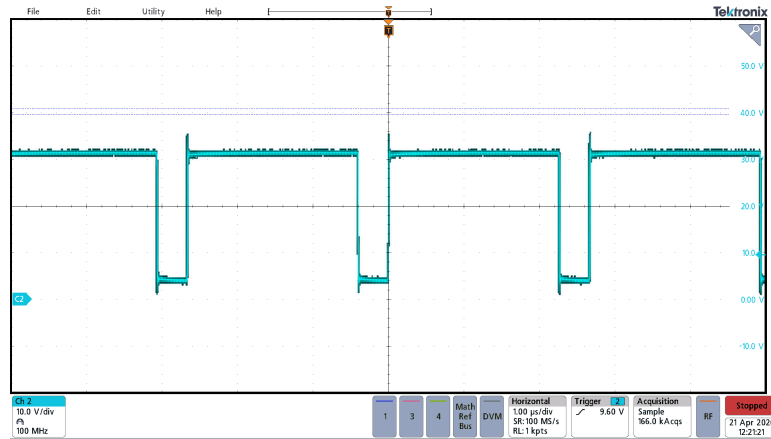


Figure 2.8: Verifying PWM with the Oscilloscope

Next, we continued with the fixed duty cycle and moved on to open loop boost testing. We connected the output to a small 10W resistive load, as shown in figure 2.9. After turning on the system, we observed that the output was 28V, which makes sense for our 85% Duty Cycle. We also noted that the inductor current stabilized to 1.5A as expected. Together, this shows a successful open loop boost test.

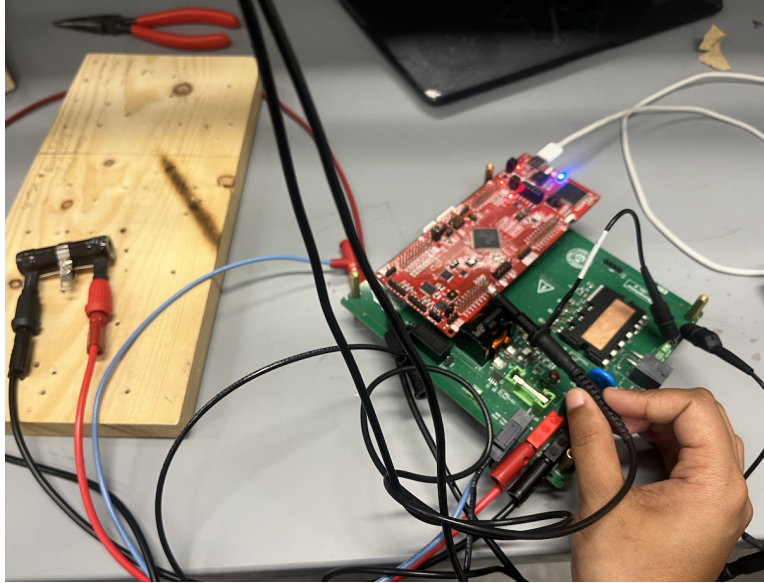


Figure 2.9: Open Loop Boost Testing: 10W Load, 85% Duty

2.2.2 Boost Closed Loop

The purpose of implementing closed loop control on the boost converter is to achieve a stable desired current. This worked for our system, as when we applied a step change to our reference current (i_{ref}) (1.5A $\rightarrow$ 1.6A), we observed that the inductor current increased to match i_{ref} . The value was already close, but we also calibrated our current sensors by adding small offsets in the code to further tune our system.

Listing 2.2: Adding Current Offsets in Software

```
// ==== ADC Scaling Factors ====
#define VIN_DIV_RHIGH 8.2e3f // ohms
#define VIN_DIV_RLOW 1e3f // ohms
#define VOUT_DIV_RHIGH 10e3f // ohms
#define VOUT_DIV_RLOW 1e3f // ohms
#define ISENSE_G_INV (1.0f / 0.055f) // A/V
#define ISENSE_OFFSET 1.646f // V, originally 1.65
```

Unfortunately, the boost converter portion of the converter stopped working at this point. We had 24V volts shorted to ground, and after multiple attempts replacing both MOSFETS and checking connections, we determined that the gate driver was the issue. It was receiving PWM, 15V, and 5V, but was not giving output switching signals. Since there were no more gate drivers left in the lab room at that point, we decided to best utilize the remaining time by moving on to the motor-focused part of the project.

2.2.3 Open Loop Motor Drive

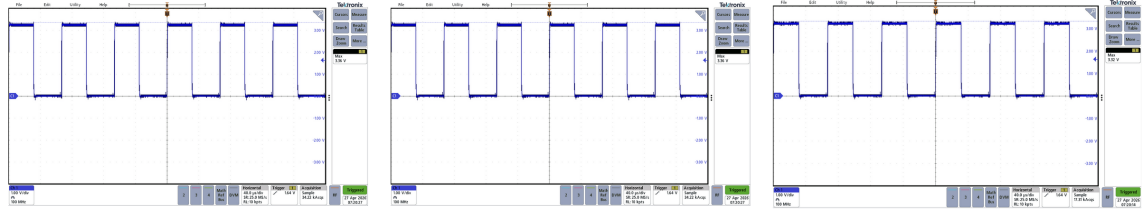


Figure 2.10: Verified PWM waveforms for each phase

Initially, we observed that the original waveform (figure 2.11) did not match our expected output. The waveform was both choppy (not a sine wave) and also out of phase. We realized that we needed to switch phase B and C in the code, which made our EMF waveform a sine wave. We also needed to change the phase offset, adding roughly 180 degrees. After fixing that, we observed the correct waveforms in phase (figure 2.12).

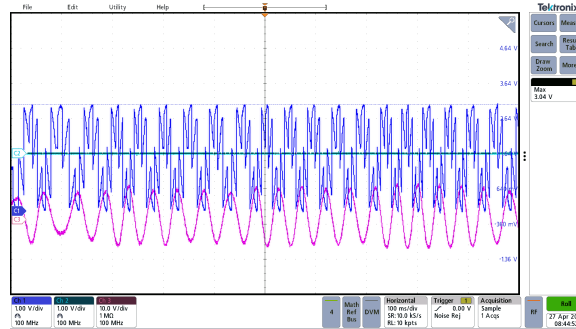


Figure 2.11: Original Back EMF Waveforms

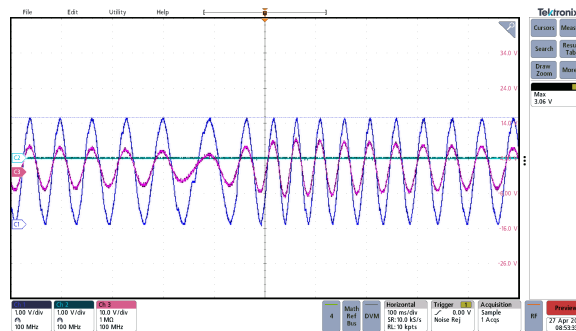


Figure 2.12: Calibrated EMF Waveforms, back in phase

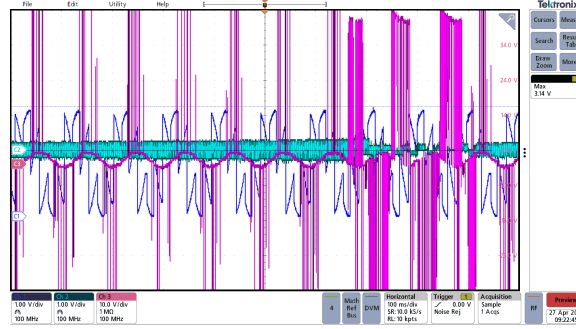


Figure 2.13: Motor Drive with Open Loop Control

The funky waveform in figure 2.13 was due to the fact that we spun the wheel counterclockwise, which was the opposite direction required. In the end, we did successfully get the wheel to spin!

2.2.4 Closed Loop Motor Control

In order to control the motor with a throttle for the full motor system, we needed to implement closed loop control to allow for feedback response.

To do so, we added update logic in ISR for inverter mode 3:

$$i_{q,ref} = throttle * I_{max}, \quad (2.1)$$

This line makes $i_{q,ref}$ proportional to the ADC value of throttle, allowing us to update the current reference the controller is aiming for, which changes the applied torque. By placing the line within the ISR, we can force this value to update regularly, allowing for real-time changes to the torque.

We also verified that the throttle was working. To do so, we fixed the motor connector wires and verified connectivity. We then connected the wires and confirmed that the watch window was capturing the change we applied to the throttle. The ADC read a value within the range of about 0.2 to 0.7, and was updating as we changed the throttle.

Unfortunately, upon running the full system, we weren't drawing current from the power supply. We tried to put everything back to get open loop running again but still no current was being drawn. After probing the IPM we noticed it was not outputting PWM to the wheel, despite all the supply pins being at the correct voltage with the correct PWM supplied by C2000. We concluded that something inside the IPM had stopped working, and because of limited time, we weren't able to fix this problem.

3 Conclusion

3.1 Discussion of Results

Our team brought the boost converter from supply verification through closed loop current step response, and ran the motor stage open loop on the bike trainer after correcting a phase-order error by swapping B and C in firmware. The closed loop iq controller and throttle-to-current mapping were implemented in the ISR and verified at the signal level, but closed loop motor tracking could not be confirmed in hardware due to a failure of the IPM late in testing.

3.2 Reflection

3.2.1 Takeaways

By working procedurally, we learned a solid framework for testing converters. Building on what we did in our power electronics course last semester, we understood the importance of working in stages and testing at checkpoints (power supply first, gate driver second, etc.) rather than just doing everything at once and having no idea what to fix when debugging issues at the end.

We have also gained understanding of how a 3-phase motor system works, from the physics of the back EMFs and inductances, to the power electronics hardware, to the dq controls. This will allow us to work effectively on similar systems in the future.

Finally, we gained a lot of hardware debugging experience. For physical testing, we were able to hone our soldering skills for both surface mount and through-hole components. We were also able to better understand the typical points of failure for a power electronic system, and how to identify what component was actually failing (for example, understanding that the IPM was broken because it was receiving all the correct signals and not outputting).

3.2.2 Future Work

If the project were extended, some next steps would be to replace the IPM and the gate driver, restore the on-board 5 V supply so that we don't have to use the C2000 as its source, refine the closed loop boost gains for better performance at even larger current step amplitudes, and confirm closed loop iq tracking against the throttle reference. With those in place, the system would be in a position to support a closed loop outer voltage loop on the boost stage and full system testing on the bike trainer under load.